

WEEK 9

Grouping data: Aggregate functions, GROUP BY, HAVING

Set operators: UNION, INTERSECT, MINUS

Aggregate Functions in SQL

Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary. **Grouping** is used to create subgroups of tuples before summarization.

Aggregate functions take a collection of values and return a single value as a result.

A number of built-in aggregate functions exist:

- **COUNT** function returns the number of tuples as specified in a query.
- **SUM** function returns sum of the values of tuples as specified in a query.
- **MAX** function returns the maximum value as specified in a query.
- **MIN** function returns the minimum value as specified in a query.
- **AVG** function returns the average value as specified in a query.

The functions SUM, MAX, MIN, and AVG can be applied to a set or multiset of numeric values. These functions can be used in the SELECT clause or in a HAVING clause. We illustrate the use of these functions with sample queries.

FNAM E	MI NIT	LNA ME	SSN	BDAT E	ADDRES S	S E X	SAL ARY	SUPER _SSN	D N O
JOHN	T	SMITH	12345 677	01/20/ 1987	GPT KGF KOLAR	M	50000	1234567 7	5
franklin	T	Smile	12345 679	06/12/ 1988	SOM KGF	M	30000	1234567 9	5
PALLA VI	C	KES HAV	12345 680	06/21/ 0091	SOM KGF	F	25000	1234568 0	4
SANTH OSH	V	KUM AR	12345 685	12/19/ 1986	GPTKAR KALA	M	55000	1234568 5	3
SUDH A	S	RANI	12345 686	02/26/ 1988	GPT KARKAL A	F	65000	1234568 6	2
SUNEE TH	S	KUM AR	12345 690	04/27/ 1988	GPT HASSAN	F	15000	1234569 0	1

Query 1. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

Q1: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
FROM EMPLOYEE;

COUNT(*)	SUM(SALARY)	MAX(SALARY)	MIN(SALARY)	AVG(SALARY)
6	240000	65000	15000	40000

Queries 2. Retrieve the total number of employees in the company

Q2: SELECT COUNT (*) FROM EMPLOYEE;

Query 3 . Retrieve the total number of employees in the ‘Research’ department .

Q3: SELECT COUNT (*)

FROM EMPLOYEE, DEPARTMENT

WHERE DNO=DNUMBER AND DNAME=‘Research’;

Here the asterisk (*) refers to the *rows* (tuples), so COUNT (*) returns the number of rows in the result of the query.

Query 4. Find the sum of the salaries of all employees of the ‘Research’ department, as well as the maximum salary, the minimum salary, and the average salary in this department.

Q4: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)

FROM EMPLOYEE, DEPARTMENT

WHERE DNO=DNUMBER AND DNAME=‘Research’;

We may also use the COUNT function to count values in a column rather than tuples, as in the next example.

Query 5. Count the number of distinct salary values in the database.

Q5: SELECT COUNT (DISTINCT Salary)

FROM EMPLOYEE;

COUNT(DISTINCTSALARY)
6

If we write COUNT(SALARY) instead of COUNT(DISTINCT SALARY) in Q5, then duplicate values will not be eliminated. However, any tuples with NULL for SALARY will not be counted. In general, NULL values are **discarded** when aggregate functions are applied to a particular column (attribute).

Grouping: The GROUP BY and HAVING Clauses

In many cases we want to apply the aggregate functions *to subgroups of tuples in a relation*, where the subgroups are based on some attribute values. In these cases we need to **partition** the relation into nonoverlapping subsets (or **groups**) of tuples. Each group (partition) will consist of the tuples that have the same value of some attribute(s), called the **grouping attribute(s)**. We can then apply the function to each such group independently to produce summary information about each group. SQL has a **GROUP BY** clause for this purpose. The GROUP BY clause specifies the grouping attributes, which should *also appear in the SELECT clause*, so that the value resulting from applying each aggregate function to a group of tuples appears along with the value of the grouping attribute(s).

Query 6. For each department, retrieve the department number, the number of employees in the department, and their average salary.

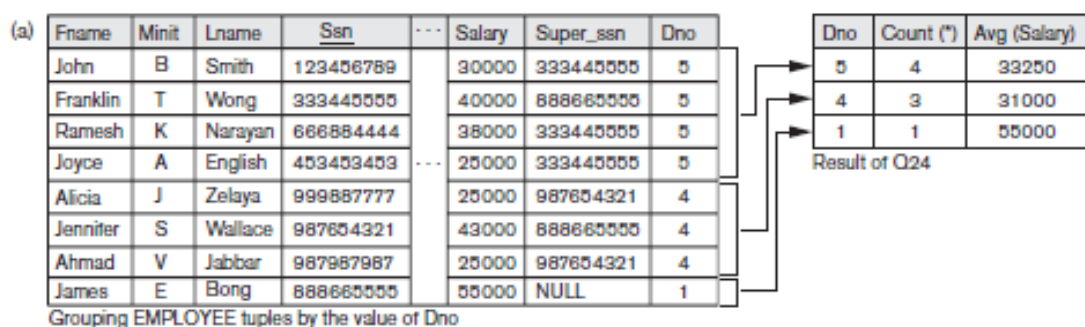
Q6: SELECT Dno, COUNT (*), AVG (Salary)

FROM EMPLOYEE

GROUP BY Dno;

In Q6, the EMPLOYEE tuples are partitioned into groups—each group having the same value for the grouping attribute Dno. Hence, each group contains the employees who work in the same department. The COUNT and AVG functions are applied to each such group of tuples. SELECT clause includes only the grouping attribute and the aggregate functions to be applied on each group of tuples.

Figure 5.1(a) illustrates how grouping works on Q6; it also shows the result of Q6.



If NULLs exist in the grouping attribute, then a **separate group** is created for all tuples with a *NULL value in the grouping attribute*. For example, if the EMPLOYEE table had some tuples that had NULL for the grouping attribute Dno, there would be a separate group for those tuples in the result of Q6.

Query 7. For each project, retrieve the project number, the project name, and the number of employees who work on that project.

Q7: SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON
WHERE Pnumber=Pno
GROUP BY Pnumber, Pname;

HAVING clause

Sometimes we want to retrieve the values of these functions only for *groups that satisfy certain conditions*. SQL provides a **HAVING** clause, which can appear in conjunction with a **GROUP BY** clause, for this purpose. **HAVING** provides a condition on the summary information regarding the group of tuples associated with each value of the grouping attributes. Only the groups that satisfy the condition are retrieved in the result of the query. This is illustrated by Query 8.

Query 8. For each project *on which more than two employees work*, retrieve the project number, the project name, and the number of employees who work on the project.

Q8: SELECT Pnumber, Pname, **COUNT** (*)
FROM PROJECT, WORKS_ON
WHERE Pnumber=Pno
GROUP BY Pnumber, Pname
HAVING **COUNT** (*) > 2;